Nand 2 Tetris

Project 3

1) Bit.hdl

Code:-CHIP Bit {

IN in, load;

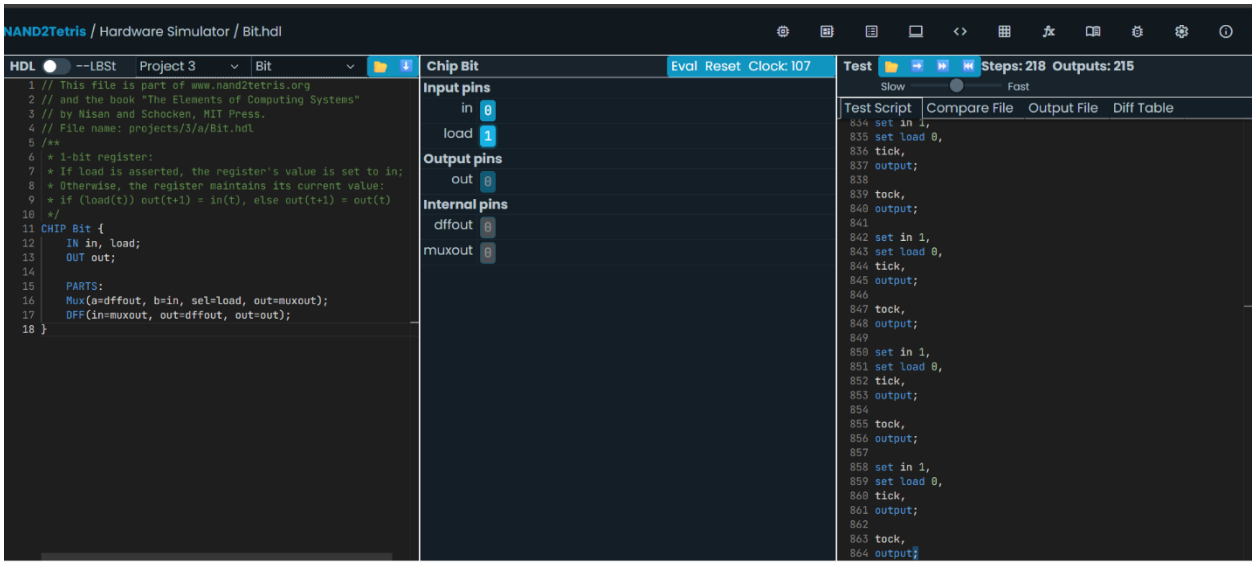
OUT out;

PARTS:

Mux(a=dffout, b=in, sel=load, out=muxout);

DFF(in=muxout, out=dffout, out=out);

}}



2) Register.hdl

Code: CHIP Register {

```
IN in[16], load;

OUT out[16];


PARTS:

Bit(in=in[0],  load=load, out=out[0]);

Bit(in=in[1],  load=load, out=out[1]);

Bit(in=in[2],  load=load, out=out[2]);

Bit(in=in[3],  load=load, out=out[3]);

Bit(in=in[4],  load=load, out=out[4]);

Bit(in=in[5],  load=load, out=out[5]);

Bit(in=in[6],  load=load, out=out[6]);

Bit(in=in[7],  load=load, out=out[7]);

Bit(in=in[8],  load=load, out=out[8]);

Bit(in=in[9],  load=load, out=out[9]);

Bit(in=in[10], load=load, out=out[10]);

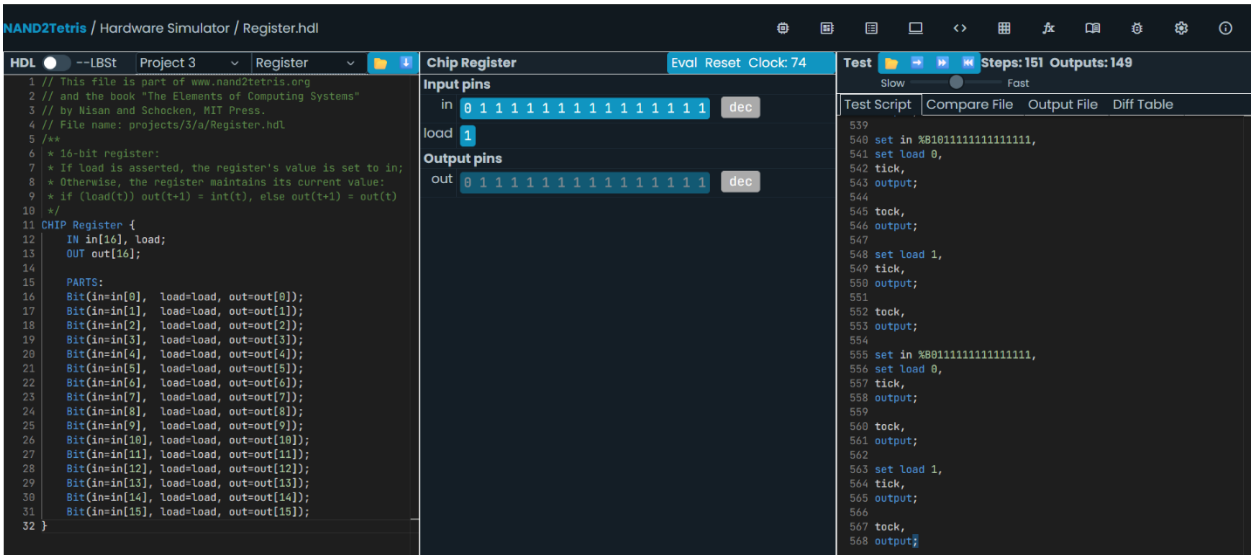
Bit(in=in[11], load=load, out=out[11]);

Bit(in=in[12], load=load, out=out[12]);

Bit(in=in[13], load=load, out=out[13]);

Bit(in=in[14], load=load, out=out[14]);

Bit(in=in[15], load=load, out=out[15]);
```


NAND2Tetris / Hardware Simulator / Register.hdl
HDL --LBSt Project 3 Register
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/3/a/Register.hdl
/**
* 16-bit register:
* If load is asserted, the register's value is set to in;
* Otherwise, the register maintains its current value:
* if (load(t)) out(t+1) = int(t), else out(t+1) = out(t)
*/
CHIP Register {
IN in[16], load;
OUT out[16];
PARTS:
Bit(in=in[0], load=load, out=out[0]);
Bit(in=in[1], load=load, out=out[1]);
Bit(in=in[2], load=load, out=out[2]);
Bit(in=in[3], load=load, out=out[3]);
Bit(in=in[4], load=load, out=out[4]);
Bit(in=in[5], load=load, out=out[5]);
Bit(in=in[6], load=load, out=out[6]);
Bit(in=in[7], load=load, out=out[7]);
Bit(in=in[8], load=load, out=out[8]);
Bit(in=in[9], load=load, out=out[9]);
Bit(in=in[10], load=load, out=out[10]);
Bit(in=in[11], load=load, out=out[11]);
Bit(in=in[12], load=load, out=out[12]);
Bit(in=in[13], load=load, out=out[13]);
Bit(in=in[14], load=load, out=out[14]);
Bit(in=in[15], load=load, out=out[15]);
}
Chip Register
Eval Reset Clock: 74
Input pins
in 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 dec
load 1
Output pins
out 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 dec
Test
Steps: 151 Outputs: 149
Slow Fast
Test Script Compare File Output File Diff Table
set in %B1011111111111111,
set load 0,
tick,
output;
tock,
output;
set load 1,
tick,
output;
tock,
output;
set in %B0111111111111111,
set load 0,
tick,
output;
tock,
output;
set load 1,
tick,
output;
tock,
output;

3)RAM8.hdl

Code:-CHIP RAM8 {

IN in[16], load, address[3];

OUT out[16];

PARTS:

DMux8Way(in=load, sel=address,

a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);

Register(in=in, load=l0, out=r0);

Register(in=in, load=l1, out=r1);

Register(in=in, load=l2, out=r2);

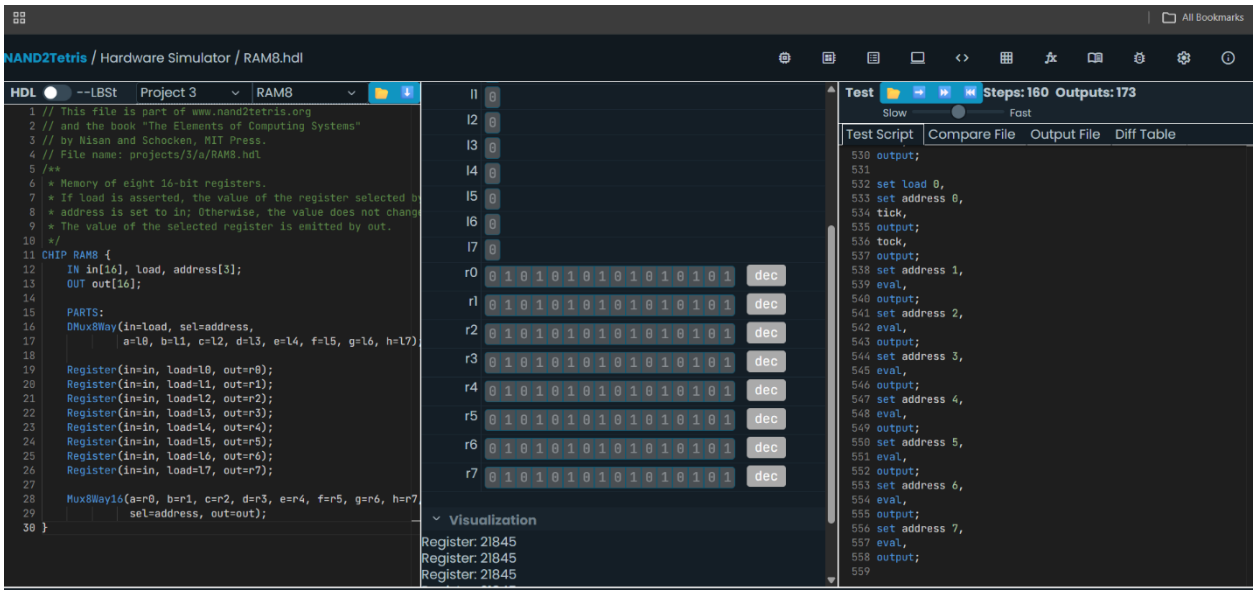
Register(in=in, load=l3, out=r3);

Register(in=in, load=l4, out=r4);

Register(in=in, load=l5, out=r5);

Register(in=in, load=l6, out=r6);

Register(in=in, load=l7, out=r7);


All Bookmarks
NAND2Tetris / Hardware Simulator / RAM8.hdl
HDL --LBSt Project 3 RAM8
// This file is part of www.nand2tetris.org
// and the book "The Elements of Computing Systems"
// by Nisan and Schocken, MIT Press.
// File name: projects/3/a/RAM8.hdl
/**
* Memory of eight 16-bit registers.
* If load is asserted, the value of the register selected b
* address is set to in; Otherwise, the value does not chang
* The value of the selected register is emitted by out.
*/
CHIP RAM8 {
IN in[16], load, address[3];
OUT out[16];
PARTS:
DMux8Way(in=load, sel=address,
a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7)
Register(in=in, load=l0, out=r0);
Register(in=in, load=l1, out=r1);
Register(in=in, load=l2, out=r2);
Register(in=in, load=l3, out=r3);
Register(in=in, load=l4, out=r4);
Register(in=in, load=l5, out=r5);
Register(in=in, load=l6, out=r6);
Register(in=in, load=l7, out=r7);
Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7
sel=address, out=out);
}
l1 0
l2 0
l3 0
l4 0
l5 0
l6 0
l7 0
r0 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
r1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
r2 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
r3 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
r4 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
r5 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
r6 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
r7 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 dec
Visualization
Register: 21845
Register: 21845
Register: 21845
Test
Steps: 160 Outputs: 173
Slow
Fast
Test Script
Compare File
Output File
Diff Table
output;
set load 0,
set address 0,
tick,
output;
tock,
output;
set address 1,
eval,
output;
set address 2,
eval,
output;
set address 3,
eval,
output;
set address 4,
eval,
output;
set address 5,
eval,
output;
set address 6,
eval,
output;
set address 7,
eval,
output;

4)RAM64.hdl

```
Code CHIP RAM64 {

    IN in[16], load, address[6];

    OUT out[16];


    PARTS:

    DMux8Way(in=load, sel=address[3..5],

            a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);


    RAM8(in=in, load=l0, address=address[0..2],
out=m0);

    RAM8(in=in, load=l1, address=address[0..2],
out=m1);

    RAM8(in=in, load=l2, address=address[0..2],
out=m2);

    RAM8(in=in, load=l3, address=address[0..2],
out=m3);

    RAM8(in=in, load=l4, address=address[0..2],
out=m4);

    RAM8(in=in, load=l5, address=address[0..2],
out=m5);

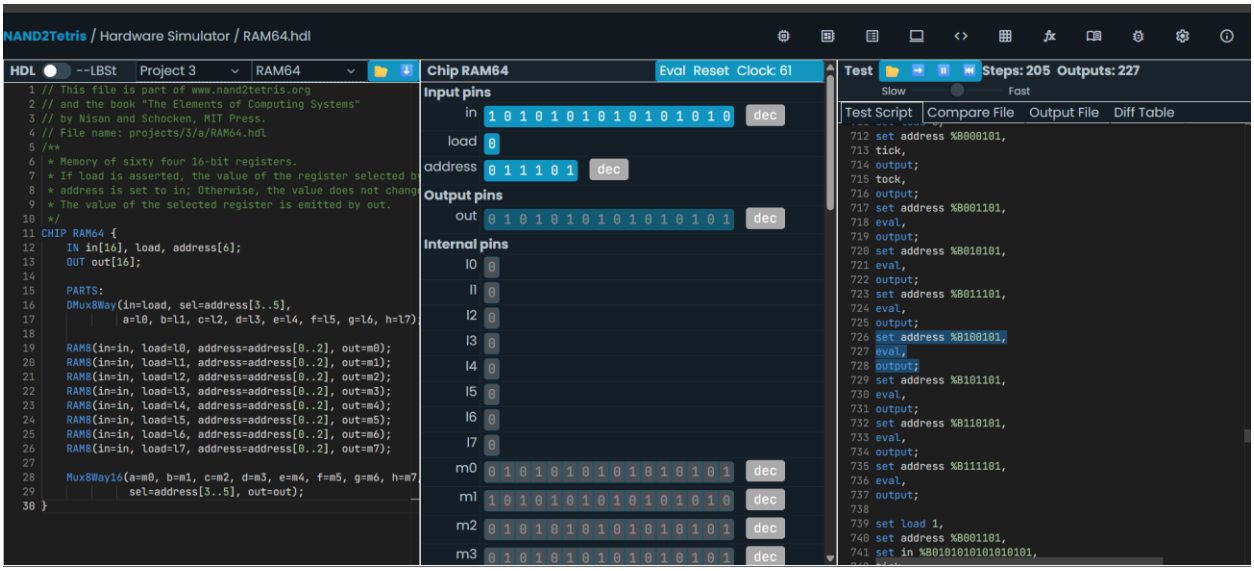
    RAM8(in=in, load=l6, address=address[0..2],
out=m6);
```

RAM8(in=in, load=l7, address=address[0..2], out=m7);

Mux8Way16(a=m0, b=m1, c=m2, d=m3, e=m4, f=m5, g=m6, h=m7,

sel=address[3..5], out=out);

}



5) PC.hdl

CODE:CHIP CHIP PC {

IN in[16], load, inc, reset;

OUT out[16];

PARTS:

Inc16(in=feedback, out=incremented);

Mux16(a=feedback, b=incremented, sel=inc, out=w1);

Mux16(a=w1, b=in, sel=load, out=w2);

Mux16(a=w2, b=false, sel=reset, out=w3);

Register(in=w3, load=true, out=out, out=feedback);

}

